FULL-STACK DEVELOPMENT (FSD) ACADEMIC PROJECT

Smart EB Connect

Location-Based Electricity Fault Reporting & Technician Assistance Web Application

A full-stack web platform designed to streamline power outage reporting and fault resolution by bridging consumers, field technicians, and system administrators.

REACT.JS + VITE | NODE + EXPRESS | MONGODB GEOJSON | LEAFLET + OSM

Problem Statement & Project Scope

Core Challenges in Fault Reporting

Traditional utility fault management suffers from opaque complaint tracking and imprecise, text-only location descriptions—causing delayed technician coordination and prolonged power outages.

Academic Prototype Scope

- **Software-Only:** Explicitly excludes ESP32 or IoT hardware sensors for automated fault detection.
- **No Official Dispatch:** Does not integrate with or authorize official government utility systems.

The Smart EB Connect Solution

A centralized web platform directly connecting Customers, Field Technicians, and Administrators. Provides real-time status tracking and geospatial proximity ranking for efficient dispatch.

⚠ Emergency Protocols & Hazards

For critical hazards (sparking, exposed wires, burning smells), users must NOT rely on the web platform.

Directly contact official emergency helplines:

1912 112

System Architecture & Technology Stack

CLIENT TIER

React 18 & Vite

Engineered for optimal performance; React Router v6 navigation.

Geospatial Integration

Leaflet + OpenStreetMap tiles; Browser Geolocation API with manual address fallback.

APPLICATION TIER

Node.js v26 & Express.js

REST API backend with Zod schema request payload validation.

Security & Auth

JWT stateless auth, bcryptjs hashing, Helmet & CORS middleware configurations.

DATA TIER

MongoDB Community 9.0

Persistence layer running natively on arm64 architecture.

Mongoose ODM & Indexing

Enforces schemas with 2dsphere GeoJSON indexes for spatial queries.

END-TO-END UNIDIRECTIONAL REQUEST FLOW

01 React UI Request

React UI initiates the HTTP request.

02 Express & RBAC

Request passes Express Routing and RBAC Middleware.

03 Controller Logic

Business logic executes in Controller with Haversine distance calculator.

04 Mongoose Translation

Mongoose translates operation into database commands.

05 MongoDB Execution

MongoDB executes query and returns document.

06 JSON Envelope

Server responds to client with standardized JSON Envelope.

Database Design & GeoJSON Schema

MongoDB collections structured to maintain data integrity and support complex geospatial queries via Mongoose ODM.

IDENTITY & AUTH

User Schema

Core Data: Full Name, unique Email, Phone Number, and bcrypt hashed password.

Role RBAC: Restricted to 'customer', 'technician', or 'admin'.

Account Flag: isActive boolean for state management.

UserSchema.js

```
email: { type: String, unique: true },
role: { enum: ['customer', ...]},
isActive: { type: Boolean }
```

FIELD WORKERS & GEO

TechnicianProfile

Link: References base User via userId.

Details: Skills array, service areas, isAvailable toggle, and aggregate rating.

Geospatial: GeoJSON Point requiring strict coordinate order [lng, lat].

GeoJSON Point Syntax

```
location: {
type: { type: String, default: 'Point' },
coordinates: [longitude, latitude]
}
```

OPERATIONAL CORE

Complaint Schema

Links: Connects customerId & assigned technicianId.

Data: problemType, description, address, & fault location GeoJSON Point.

Audit: isHazardous flag, status, & statusHistory[].

ComplaintSchema.js

```
isHazardous: { type: Boolean },
location: { type: PointSchema },
statusHistory: [StatusLogSchema]
```

Role-Based Access Control (RBAC) & User Experiences

The platform enforces strict Role-Based Access Control (RBAC) to ensure secure, context-specific user experiences tailored to three distinct user types.

CUSTOMER ROLE

Customer Workflow

Dedicated Service Portal
Portal to register, authenticate, and manage service requests.

Geospatial Fault Submission
Utilize device GPS coordinate mapping or manual address input.

Technician Selection
View ranked list of nearby technicians to request specific assignment.

Self-Service Cancellation
Cancel pending complaints if issue resolves independently.

TECHNICIAN ROLE

Technician Workflow

Mobile-Responsive Dashboard
Specialized dashboard tailored for field technicians.

GPS & Availability Toggle
Toggle active status and update real-time field GPS coordinates.

Workload Autonomy
Accept or reject assigned requests based on current workload.

Resolution Lifecycle
Progress complaints through status transitions until resolution.

ADMIN ROLE

Admin Oversight

Metrics & Platform Health
Summary metrics dashboard to monitor platform throughput.

System-Wide Complaint Audit
Filter and review all complaint logs across the entire system.

Escalation Management
Manually assign or reassign technicians for priority escalations.

User Governance
Perform critical tasks to activate or deactivate user accounts.

Geospatial Discovery & Haversine Ranking

Precision Location Capture & Data Conversion

The platform relies on accurate geospatial data to function. The frontend utilizes the HTML5 Browser Geolocation API to capture user coordinates. To prioritize privacy and prevent unauthorized tracking, this API is only invoked after an explicit user action (e.g., clicking a 'Locate Me' button). The browser API returns data in a [latitude, longitude] format; however, our application tier automatically intercepts and converts this into the strict [longitude, latitude] order required by MongoDB's GeoJSON specifications.

GEOJSON ADAPTER

`Browser [lat, lng] → MongoDB [lng, lat]`

Graceful Fallback Mechanisms

Recognizing that GPS signals can be denied by the user or unavailable due to environmental factors, the system is designed with a graceful fallback architecture. A lack of GPS data never blocks a user from submitting a critical fault report. Instead, users are seamlessly prompted to enter a manual text address or visually drop a pin on the interactive Leaflet map interface.

Haversine Formula Implementation

To optimize technician dispatch, the backend utilizes the Haversine formula to calculate distances on a sphere. This service ranks all available and active technicians, presenting them nearest-first measured in kilometers.

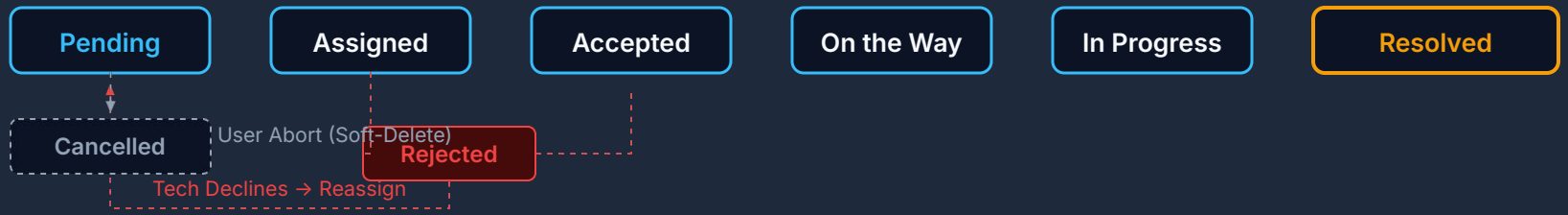
```
dispatch_order =
sort_by_distance(haversine(user_loc, tech_locs))
```

UI Distance & ETA Disclaimer

To manage user expectations accurately, the UI features a clear disclaimer explaining that this calculation represents straight-line (as-the-crow-flies) distance, which will inherently differ from actual road travel distance and estimated time of arrival (ETA).

Complaint State Machine & Concurrency Guards

Lifecycle State Machine Architecture



Race Condition Guard

`HTTP 409 ALREADY_ACCEPTED`

Actively prevents double acceptance of a fault ticket. If two technicians accept simultaneously, secondary requests are blocked.

State Enforcement

`HTTP 400 INVALID_TRANSITION`

Blocks unauthorized or illogical state jumps (e.g. Pending directly to Resolved without Acceptance).

Audit Logging

`statusHistory: [] Array`

Every lifecycle transition is permanently appended into an immutable status history array for system accountability.

REST API Overview (17 Endpoints)

The backend architecture exposes 17 distinct RESTful endpoints, logically grouped by resource and secured via RBAC middleware:

Authentication & Identity (Auth)

- POST `/api/auth/register` — Creates new users with hashed passwords
- POST `/api/auth/login` — Authenticates credentials & issues JWTs
- GET `/api/auth/me` — Retrieves current authenticated profile

Technician Operations

- GET `/api/technicians/nearby` — Geospatial queries for local techs
- PATCH `/api/technicians/me/availability` — Toggles working status
- PATCH `/api/technicians/me/location` — Updates live GPS coordinates

Complaint Management

- GET `/api/complaints` — Fetches complaint lists (role filtered)
- POST `/api/complaints` — Submits a new fault report
- GET `/api/complaints/:id` — Retrieves detailed complaint data
- PATCH `/api/complaints/:id` — Updates complaint details
- DELETE `/api/complaints/:id` — Soft-deletes/cancels a complaint
- POST `/api/complaints/:id/assign` — Assigns technician to fault
- POST `/api/complaints/:id/accept` — Technician accepts assignment
- POST `/api/complaints/:id/reject` — Technician declines assignment

Administrative & System Health

- GET `/api/health` — Returns HTTP 200 verifying server uptime
- GET `/api/admin/summary` — Aggregates system dashboard metrics
- PATCH `/api/admin/users/:id` — Modifies user account status/roles

Architecture & Middleware Highlights

- **RBAC Protection:** Route-level authorization middleware verifies user JWT roles before executing requests.
- **Geospatial Engine:** Uses MongoDB 2DSphere indexing for real-time local technician discovery.
- **State Engine:** Strict transition checks for pending fault assignment and technician dispatch.

Quality Assurance, Testing & Team Contributions

Automated Test Suite Features & 4-Member Team Division of Labor

Automated API Test Suite

To ensure enterprise-grade reliability, the backend is validated by a comprehensive automated test suite utilizing the native Node Test Runner paired with Supertest.

- **Critical Infrastructure**
  Rigorously verifies HTTP 200 health checks, Zod payload validation, and bcrypt password hashing integrity.
- **Security Boundaries**
  Explicitly tests customer isolation (verifying HTTP 403 Forbidden responses) and admin RBAC enforcement.
- **Complex Business Logic**
  Tests accuracy of the Haversine sorting algorithm and reliability of state transition concurrency guards.

Member 1 (Frontend Lead)

Architected the React UI, managed React Router navigation, integrated the Leaflet map interface, and developed role-specific dashboards.

Member 2 (Backend Logic)

Engineered Express Controllers, implemented the Haversine distance service, and enforced Zod data validation.

Member 3 (Database Architecture)

Designed Mongoose schemas, configured complex GeoJSON 2dsphere indexes, and developed database seed scripts for rapid deployment.

Member 4 (Security & QA)

Implemented JWT authentication and RBAC middleware, authored the automated Supertest suite, and managed final demo QA processes.

End-to-End Demo Walkthrough & Future Scope

Live Demonstration Flow

1. **Customer Reporting** (`ananya@smarteb.local`)
   Simulated customer logs into platform, identifies outage, and submits a geo-tagged complaint.
2. **Dispatch & Assignment** (`karthik.tech@smarteb.local`)
   System identifies & assigns nearest available field technician based on Haversine ranking.
3. **Technician Resolution**
   Technician receives alert, accepts job, travels to location, and updates state machine to 'Resolved'.
4. **Administrative Audit** (`admin@smarteb.local`)
   System administrator logs in to review updated operational metrics and audit completed ticket.

Future Scope & Enhancements

Advanced Routing
Replacing straight-line Haversine calculations with road routing & accurate ETAs via OSRM or Mapbox.

Real-Time Comms
Implementing WebSockets to push live notification updates to users without manual page refreshes.

Media Integration
Adding secure cloud storage to allow users to upload photos of faults for better technician preparation.

Accessibility
Expanding platform reach through comprehensive regional language localization.